# Multi-Agent Systems with LangGraph + Gemini

## AI Agents Workshop

---

In this notebook you'll build a **multi-agent startup analyst** from scratch.

Type a company name → a **supervisor agent** spins up 3 specialist agents in parallel → you get a full research report.

**What you'll learn:**

- What an AI agent actually is (the loop)
- How agents use **tools** to take actions
- How **LangGraph** orchestrates multiple agents
- How a **supervisor** delegates work and synthesizes results

**Stack:**

- Gemini 3 Flash - free API, fast, great for agents
- LangGraph - agent orchestration framework
- Tavily - web search API (free tier: 1000 searches/month)

**Get your free API keys before starting:**

- Gemini: https://aistudio.google.com/app/apikey
- Tavily: https://app.tavily.com/sign-in

# Step 1: Install dependencies

LangGraph handles the agent graph. `langchain-google-genai` is the Gemini connector.

```
!pip install -q langgraph langchain-google-genai langchain-community
!pip install -q langchain-tavily

# Colab needs a restart before newly installed packages are importable
import sys
if 'langchain_google_genai' not in sys.modules:
    print('Packages installed — restarting runtime (this is normal!)')
    import os; os.kill(os.getpid(), 9)
```

# 🔑 Step 2: Add your API keys

## 🔑 Step 2: Add your API keys

We use **Colab Secrets** — your key is stored privately in your Google account and never saved in the notebook.

**How to add a secret:**

1. Click the 🔑 **key icon** in the left sidebar (or go to Tools → Secrets)
2. Click **+ Add new secret**
3. Name: `GOOGLE_API_KEY` → paste your Gemini key
4. Name: `TAVILY_API_KEY` → paste your Tavily key
5. Toggle **Notebook access** on for both

Get your free keys here:

- Gemini: https://aistudio.google.com/app/apikey
- Tavily: https://app.tavily.com/sign-in

```
import os
from google.colab import userdata

# Reads from your private Colab Secrets, never touches the notebook
os.environ["GOOGLE_API_KEY"] = userdata.get("GOOGLE_API_KEY")
os.environ["TAVILY_API_KEY"] = userdata.get("TAVILY_API_KEY")

# Verify they loaded (prints True/False, never the actual key)
print("GOOGLE_API_KEY set:", bool(os.environ.get("GOOGLE_API_KEY")))
print("TAVILY_API_KEY set:", bool(os.environ.get("TAVILY_API_KEY")))
print()
print("Keys loaded from Colab Secrets. Let's build some agents.")
```

```
GOOGLE_API_KEY set: True
TAVILY_API_KEY set: True

Keys loaded from Colab Secrets. Let's build some agents.
```

## Step 3: What IS an agent?

Before we write code, here's the mental model:

```
while goal_not_achieved:
    observation = perceive(environment)   # What do I know?
    action      = think(observation)      # What should I do next?
    result      = act(action)             # Do it (call a tool)
    # loop...
```

An agent is just an LLM in a loop that can **call tools** until it decides it's done.

A **multi-agent system** is multiple agents, each with their own specialty, coordinated by a supervisor.

Let's set up the LLM brain first:

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.rate_limiters import InMemoryRateLimiter

rate_limiter = InMemoryRateLimiter(
    requests_per_second=0.3,
    check_every_n_seconds=0.1,
)

# Gemini 3 Flash, fast and free tier friendly
llm = ChatGoogleGenerativeAI(
    model="gemini-3.1-flash-lite",
    temperature=0,          # 0 = deterministic, good for agents
    rate_limiter=rate_limiter,
)

# Quick Test
response = llm.invoke("Say 'agents are cool' and nothing else.")
print(response.content)
```

```
[{'type': 'text', 'text': 'agents are cool', 'extras': {'signature': '
```

## Step 4: Give agents tools

Without tools, an agent can only *think*. Tools let it *act* — search the web, run code, call APIs.

We'll use **Tavily** for web search. It's built for LLM agents — returns clean, summarized results.

```
from langchain_tavily import TavilySearch
from langchain_core.tools import tool

# Web search tool -> returns top 5 results
web_search = TavilySearch(max_results=5)

# A focused news search tool
# Note: @tool needs docstring
@tool
def news_search(query: str) -> str:
```

```
    """Search for recent news about a company or topic."""
    results = TavilySearch(max_results=5).invoke(f"latest news {query}
    return str(results)

@tool
def competitor_search(company: str) -> str:
    """Find competitors and alternatives to a given company."""
    results = TavilySearch(max_results=5).invoke(f"{company} competit
    return str(results)

@tool
def sentiment_search(company: str) -> str:
    """Search for public sentiment, reviews, and opinions about a com
    results = TavilySearch(max_results=5).invoke(f"{company} reviews
    return str(results)

print("Tools ready!")
print(f"  - web_search: general web search")
print(f"  - news_search: recent news")
print(f"  - competitor_search: find rivals")
print(f"  - sentiment_search: public opinion")
```

```
Tools ready!
  - web_search: general web search
  - news_search: recent news
  - competitor_search: find rivals
  - sentiment_search: public opinion
```

## Step 5: Build the specialist agents

Each agent is an LLM + a system prompt defining its role + specific tools.

Think of them like employees with different job descriptions.

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceh
from langgraph.prebuilt import create_react_agent

def make_agent(system_prompt: str, tools: list, name: str):
    """Factory function: creates a named agent with a role and toolse
    return create_react_agent(
        model=llm,
        tools=tools,
        prompt=system_prompt,
        name=name,
    )

# Researcher: gets the basics on the company
researcher = make_agent(
    system_prompt="""You are a business researcher. Your job is to fi
```

```
        what they do, when they were founded, their business model, fundi
        Be concise. Return structured facts, not prose.""",
        tools=[web_search, news_search],
        name="Researcher"
    )

    # Competitor Analyst
    competitor_analyst = make_agent(
        system_prompt="""You are a competitive intelligence analyst. Your
        the main competitors of a company, how they compare on features/p
        and what the competitive moat (if any) looks like. Be specific.""
        tools=[competitor_search, web_search],
        name="CompetitorAnalyst"
    )

    # Sentiment Analyst
    sentiment_analyst = make_agent(
        system_prompt="""You are a brand sentiment analyst. Your job is t
        think about a company — common praise, common complaints, overall
        and any notable controversies. Use reviews, Reddit, forums.""",
        tools=[sentiment_search, web_search],
        name="SentimentAnalyst"
    )

    print("Three specialist agents created:")
    print("  - Researcher")
    print("  - CompetitorAnalyst")
    print("  - SentimentAnalyst")
```

```
Three specialist agents created:
  - Researcher
  - CompetitorAnalyst
  - SentimentAnalyst
/tmp/ipykernel_11684/1488745382.py:6: LangGraphDeprecatedSinceV10: cre
  return create_react_agent(
```

## Step 6: Build the supervisor with LangGraph

The supervisor is the orchestrator. It:

1. Receives the user's question
2. Decides which agents to call and in what order
3. Collects their results
4. Synthesizes a final report

LangGraph models this as a **state graph** — nodes are agents, edges are the flow between them.

```
User → Supervisor → [Researcher, CompetitorAnalyst, SentimentAnalyst] → S
```

```
from typing import TypedDict, Annotated, Sequence
from langgraph.graph import StateGraph, END
from langchain_core.messages import BaseMessage, HumanMessage, AIMessa
import operator

# State: shared memory passed between all agents
class AgentState(TypedDict):
    company: str                                          # The compar
    messages: Annotated[list[BaseMessage], operator.add]  # Full conv
    research_result: str                                  # Output fr
    competitor_result: str                                # Output fr
    sentiment_result: str                                 # Output fr
    final_report: str                                     # Superviso


# Node functions: each runs one agent and updates state

def run_researcher(state: AgentState) -> dict:
    print("\n [Researcher] Starting research...")
    result = researcher.invoke({"messages": [("user", f"Research this
    print(f" [Researcher] Done.")
    return {"research_result": result["messages"][-1].content}

def run_competitor_analyst(state: AgentState) -> dict:
    print("\n [CompetitorAnalyst] Mapping competitive landscape...")
    result = competitor_analyst.invoke({"messages": [("user", f"Analy
    print(f" [CompetitorAnalyst] Done.")
    return {"competitor_result": result["messages"][-1].content}

def run_sentiment_analyst(state: AgentState) -> dict:
    print("\n [SentimentAnalyst] Reading public sentiment...")
    result = sentiment_analyst.invoke({"messages": [("user", f"Analys
    print(f" [SentimentAnalyst] Done.")
    return {"sentiment_result": result["messages"][-1].content}

def run_supervisor(state: AgentState) -> dict:
    print("\n🕵️ [Supervisor] Synthesizing all results into final repo

    synthesis_prompt = f"""
    You are a senior analyst. Synthesize the following research into a

    Company: {state['company']}

    === RESEARCH ===
    {state.get('research_result', 'N/A')}

    === COMPETITIVE LANDSCAPE ===
```

```
    {state.get('competitor_result', 'N/A')}

    === PUBLIC SENTIMENT ===
    {state.get('sentiment_result', 'N/A')}

    Write a well-structured markdown report with sections:
    1. Company Overview
    2. Competitive Landscape
    3. Public Sentiment
    4. Key Takeaways (3-5 bullet points)
    """

    response = llm.invoke(synthesis_prompt)
    return {"final_report": response.content}


print("Node functions defined.")
```

```
Node functions defined.
```

```
from langgraph.graph import StateGraph, END

# Build the graph
graph = StateGraph(AgentState)

# Add each agent as a node
graph.add_node("researcher",          run_researcher)
graph.add_node("competitor_analyst",  run_competitor_analyst)
graph.add_node("sentiment_analyst",   run_sentiment_analyst)
graph.add_node("supervisor",          run_supervisor)

# Entry point: researcher runs first
graph.set_entry_point("researcher")

# After researcher, run both analyst agents
graph.add_edge("researcher",         "competitor_analyst")
graph.add_edge("researcher",         "sentiment_analyst")

# Both analysts feed into supervisor
graph.add_edge("competitor_analyst", "supervisor")
graph.add_edge("sentiment_analyst",  "supervisor")

# Supervisor is the end
graph.add_edge("supervisor", END)

# Compile the graph into a runnable
app = graph.compile()

print("Agent graph compiled!")
print()
```

```
print("Flow: User → Researcher → [CompetitorAnalyst + SentimentAnalys
```

```
Agent graph compiled!

Flow: User → Researcher → [CompetitorAnalyst + SentimentAnalyst] → Sup
```

## Step 7: Run it!

Type any company name and watch the agents work.

> Watch the output as it streams in. You'll see each agent's `Thought → Action → Observation` loop printed in real time. That's the agent reasoning.

```
# Change this to any company you want to research!
COMPANY = "Nebius"

print(f"Launching multi-agent analysis for: {COMPANY}")
print("=" * 60)
print()

# Run the graph
result = app.invoke({
    "company": COMPANY,
    "messages": [HumanMessage(content=f"Analyse {COMPANY}")],
    "research_result": "",
    "competitor_result": "",
    "sentiment_result": "",
    "final_report": "",
})

print()
print("=" * 60)
print(f"Analysis complete!")
```

```
Launching multi-agent analysis for: Nebius
============================================================


 [Researcher] Starting research...
 [Researcher] Done.

 [CompetitorAnalyst] Mapping competitive landscape...

 [SentimentAnalyst] Reading public sentiment...
 [CompetitorAnalyst] Done.
 [SentimentAnalyst] Done.

 [Supervisor] Synthesizing all results into final report
```

```
✍️ [Supervisor] Synthesizing all results into final report...

============================================================
Analysis complete!
```

## Step 8: Read the report

```
from IPython.display import Markdown, display

report = result["final_report"]
if isinstance(report, list):
    report = report[0]["text"]

display(Markdown(report))
```

# Executive Analysis: Nebius Group (NASDAQ: NBIS)

## 1. Company Overview

Nebius Group N.V. is an AI-native cloud infrastructure provider headquartered in Amsterdam. Following its 2024 corporate restructuring and separation from its former Russian operations (Yandex), the company has emerged as a specialized player in the high-performance computing (HPC) market.

- **Core Business:** Provides Infrastructure-as-a-Service (IaaS) and Platform-as-a-Service (PaaS) optimized for AI model training and deployment.
- **Strategic Focus:** Offers a full-stack AI cloud, including GPU-accelerated compute, specialized storage, and networking designed for massive AI clusters.
- **Financial Position:** Recently secured ~$4.34 billion in convertible debt to fund aggressive expansion of data centers and GPU capacity.
- **Portfolio:** Beyond its core cloud business, the group retains ownership of other ventures, including the autonomous driving company **Avride**.

## 2. Competitive Landscape

Nebius operates in a high-stakes market, competing against both specialized GPU clouds and massive hyperscalers.

| Tier | Competitors | Positioning |
|---|---|---|
| **Specialized AI Clouds** | CoreWeave, Lambda Labs, RunPod | Direct rivals; compete on GPU availability and time-to-market. |
| **Hyperscalers** | AWS, Google Cloud, Azure | Indirect rivals; compete on ecosystem lock-in and integrated services. |
| **Sovereign Clouds** | Regional European providers | Focus on data residency and GDPR compliance. |

**Competitive Moat:**

- **Engineering DNA:** Leverages a deep-tech engineering team with extensive experience in distributed systems and large-scale infrastructure.
- **Vertical Integration:** Proprietary software stacks allow for performance optimizations (e.g., lower latency) that generic providers cannot match.
- **Strategic Expansion:** Recent acquisitions (e.g., Tavily) signal a shift toward offering "AI-ready" software services, moving the company up the value chain.
- **Geopolitical Positioning:** Its European headquarters and focus on "sovereign AI" provide a regulatory advantage for enterprises concerned with US-based cloud privacy laws.

### 3. Public Sentiment

Public perception of Nebius is currently bifurcated, reflecting its status as a "new" entity in the Western market.

- **Investor Sentiment (Bullish):** Driven by the company's aggressive infrastructure scaling (e.g., the 1.2 GW Missouri project) and its status as a pure-play AI infrastructure stock.
- **Technical/User Sentiment (Cautiously Curious):** Developers praise the "S-tier" engineering talent and the platform's AI-native architecture. However, there is skepticism regarding long-term reliability and support compared to established

## What just happened?

Let's break down what the system did:

1. **You** gave it a company name — that's the only input
2. **LangGraph** started the graph at the `researcher` node
3. **Researcher agent** decided on its own to call `web_search` and `news_search`, read the results, and returned a summary
4. **LangGraph** then ran `competitor_analyst` and `sentiment_analyst` (both get the researcher's output via shared state)
5. Each specialist agent used its own tools to do its job independently
6. **Supervisor** received all three outputs and synthesized the final report — no instructions needed

That's a multi-agent system. Each agent:

- Has a **role** (system prompt)
- Has **tools** it can use
- **Reasons autonomously** about what to do
- **Reports back** to the orchestrator

## Challenges: try these yourself!

**Easy:**

- Change `COMPANY` to a different startup and re-run
- Add a new tool (e.g. a `funding_search` that looks for funding rounds)

**Medium:**

- Add a 4th specialist agent: a **`FinanceAgent`** that looks up revenue, valuation, and funding
- Add a `human_in_the_loop` node that pauses and asks you to approve the report before finalizing

**Hard:**

- Make the supervisor **dynamic** it decides which agents to call based on the query (not always all 3)
- Add **memory** so the agent remembers companies it has researched before
- Connect a real MCP server (try the Brave Search MCP)

---

**Resources:**

- LangGraph docs: https://langchain-ai.github.io/langgraph/
- LangGraph tutorials: https://langchain-ai.github.io/langgraph/tutorials/
- MCP protocol: https://modelcontextprotocol.io
- Gemini API: https://ai.google.dev/
- Tavily (search API): https://tavily.com